

Trabalho 2 - Sistemas Operacionais (INF1316)

Gerência de Memória - Simulador de Memória Virtual

1. Identificação

Nome: Caio Faria Brito Martins Chaves

Matrícula: 2410162

Nome: Lucas Hufnagel Gromann de Araujo Goes

Matrícula: 2410845

Implementação do simulador e execução das simulações: ambos os integrantes. Os quatro algoritmos de substituição (LRU, NRU, Relógio e Ótimo) foram implementados em conjunto.

2. Objetivo

O objetivo deste trabalho é implementar, em linguagem C, um simulador de memória virtual com paginação (sim-virtual), que faça o mapeamento de endereços lógicos em físicos e trate a substituição de páginas por meio de quatro algoritmos: LRU, NRU, Algoritmo do Relógio e Algoritmo Ótimo.

O simulador recebe como entrada um arquivo com a sequência de endereços de memória acessados por um programa real (em hexadecimal, seguidos de R ou W). A cada acesso ele atualiza os bits de controle das páginas, detecta as faltas de página (page faults) e, quando a memória física está cheia, simula a substituição de uma página por outra. Ao final, gera um pequeno relatório com a configuração usada, o número de faltas de página e o número de páginas “suja” que precisariam ser escritas de volta no disco.

Além da implementação, o trabalho teve como finalidade:

- calcular o índice da página a partir do endereço lógico, para diferentes tamanhos de página;
- manter os bits R (referenciada) e M (modificada) e o instante do último acesso de cada página em memória;
- implementar os quatro algoritmos de substituição e contar, em cada um, os page faults e as escritas de páginas sujas;
- executar todas as combinações pedidas (página de 4 KB e 8 KB; memória de 1, 2 e 4 MB) para os quatro programas fornecidos;
- comparar o desempenho dos algoritmos e justificar os valores obtidos.

3. Estrutura do Programa

O programa foi organizado em dois arquivos principais, sim-virtual.c e sim-virtual.h, acompanhados de um Makefile e de um script auxiliar (run_all.sh) usado para rodar todas as simulações de uma vez e gerar a tabela de resultados.

3.1 Do endereço lógico para a página

O índice da página é obtido descartando os s bits menos significativos do endereço, ou seja, $page = addr \gg s$. O valor de s é calculado a partir do tamanho da página passado na linha de comando: como a página é uma potência de 2, basta contar quantas vezes ela pode ser dividida por 2. Assim, para páginas de 4 KB temos $s = 12$ e para 8 KB, $s = 13$. O número de quadros da memória física é dado por $num_quadros = (memória \text{ em bytes}) / (tamanho \text{ da página em bytes})$.

3.2 Estruturas de dados

As estruturas usadas estão definidas em `sim-virtual.h`:

- **Quadro**: representa um quadro da memória física. Guarda a página que está nele (ou -1 se estiver vazio), o bit R, o bit M e o instante do último acesso. Esses são exatamente os campos de controle pedidos no enunciado e atendem a todos os algoritmos.
- **Tabela de páginas**: é um vetor indexado pelo número da página que diz em qual quadro aquela página está (ou -1, se não estiver na memória). Como os endereços têm 32 bits, essa tabela pode ter muitas entradas (até cerca de 1 milhão para páginas de 4 KB); como estamos simulando um único programa, isso não é um problema e dá busca em tempo constante.
- **Vetor de acessos**: a sequência inteira do arquivo `.log` é carregada em memória. Isso é necessário para o algoritmo Ótimo, que precisa “olhar o futuro”, e é usado de forma uniforme pelos demais algoritmos.

Um contador de tempo, iniciado em 0 e incrementado a cada acesso, é usado para registrar o momento de cada referência. As faltas de página são contadas toda vez que uma página é carregada na memória (inclusive no preenchimento inicial dos quadros) e as escritas são contadas apenas quando uma página modificada é removida. Páginas sujas que ainda estão na memória no fim da execução não são contadas, conforme o enunciado.

3.3 Os algoritmos de substituição

- **LRU (Least Recently Used)**: a cada acesso atualiza o instante do último acesso do quadro. Quando precisa substituir, escolhe a página com o menor instante, ou seja, a que está há mais tempo sem ser usada.
- **NRU (Not Recently Used)**: separa as páginas em quatro classes a partir dos bits (R, M) e remove uma página da menor classe não vazia. Para o bit R refletir o uso recente, ele é zerado de tempos em tempos (a cada 1000 acessos), imitando o tick do relógio do sistema operacional.
- **Relógio (segunda chance)**: trata os quadros como uma lista circular com um ponteiro. Se a página apontada tem $R = 1$, ela ganha uma segunda chance (zera R e o ponteiro avança); se tem $R = 0$, é a página escolhida. É uma aproximação mais barata do LRU.
- **Ótimo**: remove a página cujo próximo uso está mais distante no futuro (ou que não será mais usada). É impossível de implementar na prática porque depende de conhecer o futuro, mas serve como limite inferior de faltas para comparação. Para deixá-lo eficiente, pré-calculamos, para cada página, a lista ordenada dos instantes em que ela é acessada; assim o próximo uso de qualquer página sai em tempo constante.

3.4 Pseudo-código do algoritmo Ótimo

```
função Ótimo(acessos A[0..n-1], num_quadros):  
    # para cada página, guarda em ordem os instantes em que ela é acessada  
    para i de 0 até n-1:  
        p = página(A[i]); guarda i na lista de ocorrências de p  
  
    quadros = {}; faltas = 0; escritas = 0  
    para tempo de 0 até n-1:  
        p = página(A[tempo]); escrita = (A[tempo] é 'W')  
        avança a lista de p (descartando o instante atual) # passa a apontar o próximo uso  
  
        se p já está nos quadros:  
            marca R (e M, se escrita) de p          # acerto, não é falta  
            continua  
  
        faltas = faltas + 1                            # page fault  
        se há quadro livre:  
            carrega p em um quadro livre  
        senão:  
            vítima = página com o MAIOR próximo uso # lista vazia = nunca mais usada  
            se a vítima está suja (M = 1): escritas = escritas + 1  
            remove a vítima e carrega p no lugar dela  
    retorna (faltas, escritas)
```

4. Solução e Resultados

Cada um dos quatro programas fornecidos (compilador, matriz, compressor e simulador), com 1.000.000 de acessos cada, foi simulado para todas as combinações de algoritmo (LRU, NRU, Relógio e Ótimo), tamanho de página (4 KB e 8 KB) e tamanho de memória (1, 2 e 4 MB), totalizando 24 simulações por programa e 96 no total. Essa varredura foi feita pelo script `run_all.sh`, que executa o simulador em todas as combinações e salva os resultados em `resultados.csv`. As tabelas abaixo mostram, para cada programa, o número de page faults (PF) e de páginas escritas (PE).

4.1 Programa: compilador.log

Pág	Mem	LRU PF	LRU PE	NRU PF	NRU PE	Relógio PF	Relógio PE	Otimo PF	Otimo PE
4	1	25.308	4.231	41.819	3.379	26.789	4.528	12.667	2.483
4	2	10.425	2.173	38.178	2.282	11.472	2.398	5.662	1.328
4	4	4.391	955	22.559	239	4.765	1.086	3.047	726
8	1	36.707	5.775	41.975	4.464	38.827	6.250	21.271	3.711
8	2	21.091	3.839	38.805	3.313	22.655	4.162	10.118	2.190
8	4	7.632	1.756	32.604	1.687	8.412	1.981	4.232	1.078

4.2 Programa: matriz.log

Pág	Mem	LRU PF	LRU PE	NRU PF	NRU PE	Relógio PF	Relógio PE	Otimo PF	Otimo PE
4	1	5.673	1.866	16.410	1.901	5.980	1.973	3.572	1.267
4	2	3.632	1.159	13.863	1.283	3.795	1.218	2.672	985
4	4	2.803	789	9.133	276	2.901	809	2.543	672
8	1	10.928	3.273	17.373	2.470	12.896	3.844	5.361	1.827
8	2	4.745	1.623	14.764	1.652	4.870	1.676	2.969	1.109
8	4	2.950	985	12.175	1.072	3.064	1.011	2.295	843

4.3 Programa: compressor.log

Pág	Mem	LRU PF	LRU PE	NRU PF	NRU PE	Relógio PF	Relógio PE	Otimo PF	Otimo PE
4	1	397	48	595	0	432	71	317	36
4	2	317	0	317	0	317	0	317	0
4	4	317	0	317	0	317	0	317	0
8	1	533	132	916	18	577	163	345	86
8	2	255	0	255	0	255	0	255	0
8	4	255	0	255	0	255	0	255	0

4.4 Programa: simulador.log

Pág	Mem	LRU PF	LRU PE	NRU PF	NRU PE	Relógio PF	Relógio PE	Otimo PF	Otimo PE
4	1	11.240	4.092	25.906	5.127	11.876	4.319	5.981	2.575
4	2	5.823	2.444	24.154	4.385	6.119	2.587	4.125	1.884
4	4	4.468	1.846	18.277	2.146	4.618	1.937	3.890	1.565
8	1	16.479	5.546	24.885	6.002	17.923	6.070	9.179	3.528
8	2	8.556	3.395	22.747	4.616	9.140	3.666	4.748	2.160
8	4	4.732	2.094	20.944	3.952	4.875	2.171	3.498	1.620

PF = page faults (páginas carregadas). PE = páginas escritas (páginas sujas removidas).

5. Análise de desempenho

5.1 Comparação entre os algoritmos

Em todas as 96 simulações o algoritmo Ótimo apresentou o menor número de page faults, como era esperado, já que ele é o limite inferior teórico. Por exemplo, no compilador com página de 4 KB e memória de 1 MB, o Ótimo teve 12.667 faltas, contra 25.308 do LRU, 26.789 do Relógio e 41.819 do NRU — praticamente metade das faltas do LRU.

Entre os algoritmos que dá para implementar de verdade, o LRU e o Relógio ficaram sempre muito próximos, com o Relógio um pouco pior. Isso faz sentido, porque o Relógio é justamente uma aproximação do LRU usando só o bit R. O NRU foi consistentemente o que mais gerou faltas: como ele separa as páginas em apenas quatro classes e depende do momento em que os bits R são zerados, acaba sendo o mais “grosseiro” dos quatro.

Por outro lado, o NRU foi o que menos escreveu páginas sujas de volta ao disco, porque prefere remover páginas que não foram modificadas (classes 0 e 2 antes das classes 1 e 3). O caso mais claro é o compilador com 4 KB e 4 MB: o NRU escreveu apenas 239 páginas, contra 955 do LRU e 1.086 do Relógio. Ou seja, o NRU troca mais faltas de página por menos escritas no disco.

5.2 Variando o tamanho da página com a memória fixa

Mantendo a memória física fixa e dobrando o tamanho da página (de 4 para 8 KB), o número de quadros cai pela metade, mas cada página passa a cobrir o dobro do espaço de endereços. O efeito final depende da localidade de cada programa:

- Para o compilador, o matriz e o simulador, o número de page faults aumentou com páginas de 8 KB. No matriz com LRU e 1 MB, por exemplo, as faltas passaram de 5.673 (4 KB) para 10.928 (8 KB); no compilador, de 25.308 para 36.707. Nesses casos, perder metade dos quadros pesou mais do que o ganho de cobertura, ou seja, a localidade espacial não foi forte o suficiente para compensar — mesmo no matriz, cujo acesso às matrizes não é suficientemente sequencial dentro de uma página maior.
- Já o compressor tem um conjunto de trabalho pequeno, que cabe na memória a partir de 2 MB. Nesse ponto as faltas passam a ser só as obrigatórias (primeira carga de cada página) e, como páginas maiores significam menos páginas distintas, o tamanho de 8 KB gerou menos faltas (255 contra 317, em 2 MB e em 4 MB).

Quanto às escritas, páginas maiores tendem a aumentar o número de páginas sujas removidas, porque uma página maior tem mais chance de conter algum endereço que foi escrito. No compilador com LRU e 4 MB, por exemplo, as escritas subiram de 955 (4 KB) para 1.756 (8 KB).

5.3 Variando o tamanho da memória

Aumentar a memória física (mais quadros) sempre reduziu o número de page faults, em todos os algoritmos e programas. No compilador com LRU e página de 4 KB, as faltas caíram de 25.308 (1 MB) para 10.425 (2 MB) e 4.391 (4 MB). Quando o conjunto de trabalho inteiro passa a caber na memória (como no compressor a partir de 2 MB), sobram apenas as faltas obrigatórias e o número de escritas vai a zero, já que nenhuma página precisa mais ser removida.

6. Observações e Conclusões

Dificuldades

- calcular o valor de s a partir do tamanho da página, de forma que o simulador funcionasse tanto para 4 KB quanto para 8 KB;
- implementar o algoritmo Ótimo de maneira eficiente, já que olhar o futuro a cada substituição seria muito lento — a solução foi pré-calcular as ocorrências de cada página;
- decidir corretamente o que contar como falta e como escrita (em especial, lembrar que páginas sujas ainda residentes no fim não contam como escrita);
- no NRU, entender que o bit R precisa ser zerado de tempos em tempos para o algoritmo fazer sentido.

Resultados observados

- o Ótimo teve sempre as menores faltas, confirmando seu papel de limite inferior;
- LRU e Relógio ficaram bem próximos, com o Relógio um pouco pior, e o NRU foi o que mais faltou, porém o que menos escreveu no disco;
- para estes programas, páginas maiores quase sempre pioraram as faltas, exceto quando o conjunto de trabalho já cabia na memória (compressor);
- aumentar a memória sempre reduziu as faltas.

Conclusão final

O trabalho permitiu observar, na prática, como diferentes algoritmos de substituição se comportam diante de cargas reais de acesso à memória. Os resultados ficaram de acordo com a teoria: Ótimo melhor que LRU e Relógio, que por sua vez são melhores que o NRU em número de faltas; o NRU compensando com menos escritas no disco; e mais memória sempre ajudando. Entre os algoritmos realizáveis, o LRU foi o melhor na maioria dos cenários, com o Relógio como uma alternativa de implementação mais simples e desempenho praticamente equivalente. A comparação também mostrou que aumentar o tamanho da página nem sempre ajuda: quando o conjunto de trabalho não cabe na memória, ter menos quadros acaba prejudicando o desempenho.

7. Como compilar e executar

```
make                # compila o sim-virtual
./sim-virtual LRU compilador.log 8 2

# para rodar todas as combinações de uma vez (com os .log no diretório):
./run_all.sh        # gera o arquivo resultados.csv
```

Para validar a implementação, as saídas dos quatro algoritmos foram comparadas com uma implementação de referência feita à parte; o número de page faults e de páginas escritas coincidiu em todos os casos testados.